

SW개발/HW제작 설계 서

프로젝트 명 : 교통사고 예방을 위한 노약자
보행 보조차 기능 개선

2024. 07. 17

노약자를 위한 스마트 보조차 팀

수행 단계별 주요 산출물

단계	산출물	일반	응용 소프트웨어	응용 하드웨어
		·모바일 APP ·Web 등	·빅데이터 ·인공지능 ·블록체인 등	·IoT ·로봇 ·드론 등
환경 분석	시장/기술 환경 분석서	△	△	△
	설문조사 결과서	-	-	-
	인터뷰 결과서	-	-	-
요구사항 분석	요구사항 정의서	○	○	○
	유즈케이스 정의서	△	△	△
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○
	서비스 흐름도(데이터 흐름도)	△	○	△
	UI/UX 정의서	-	-	-
	하드웨어/센서 구성도	-	-	○
기능 설계	메뉴 구성도	○	○	○
	화면 설계서	○	○	△
	엔티티 관계도	-	-	-
	기능 처리도(기능 흐름도)	○	○	○
	알고리즘 명세서/설명서	△	○	○
	데이터 수집처리 정의서	-	-	-
	하드웨어 설계도	-	-	○
개발 / 구현	프로그램 목록	△	△	△
	테이블 정의서	-	-	-
	핵심 소스코드	○	○	○

※ ○ 필수, △ 선택

| 시장/기술 동향 분석



출처: 도로교통공단, 교통사고분석시스템(TAAS)

왼쪽은 전체 보행자 교통사고 수와 그 중 고령 보행자 사고수 및 비율을 보여주는 그래프.
오른쪽은 전체 보행자 사망 수와 그 중 고령 보행자 사망자수 및 비율을 보여주는 그래프.

위 자료에 따르면 전체 보행자 사고 건수와 사망자는 2019년도부터 매년 조금씩 줄어드는 추세에 비해 고령 보행자 사고 건수와 사망자 수는 증가하며 그로 인한 비율은 더욱 상승하는 모습을 나타냅니다.

특성별(1)	특성별(2)	2022.1/2	
		건수 (건)	분율 (%)
전체	소계	17,668	100.0
성별	남자	11,212	63.5
	여자	6,453	36.5
	미상	-	-
연령별	0-9세	168	1.0
	10-19세	164	0.9
	20-29세	416	2.4
	30-39세	567	3.2
	40-49세	1,179	6.7
	50-59세	2,253	12.8
	60-69세	3,179	18.0
	70-79세	3,932	22.3
	80세 이상	5,809	32.9

2022년도 상반기 전체 급성심장정지 건수와 성별, 연령별 수 및 비율을 보여주는 KOSIS 통계표.

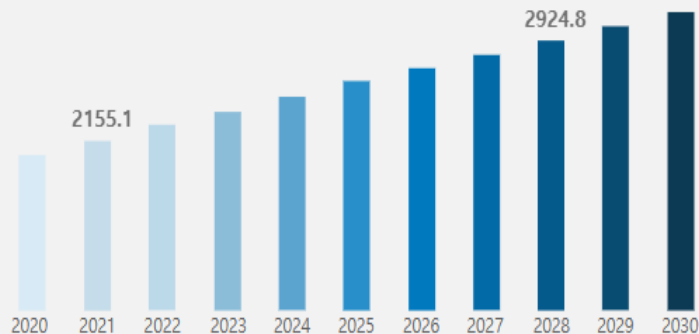
KOSIS 통계를 따르면 노약자들이 길을 걷다가 갑자기 쓰러지는 건강 이상 증세 급성 심장정지 사고가 50대 이상이 86%로 대부분을 차지합니다.

또한 목격 여부에 따른 급성심장정지 사고 통계에서도 목격된 사고는 생존율이 12%이지만, 목격되지 않았을 경우 생존율은 3%에 불과합니다. 이런 사고로 발생하는 2차 교통사고가 일어날 가능성이 있습니다.

시장/기술 동향 분석

출처: Business Researcher
Insight

Global Medical Walkers Market Size, 2031 (USD Million)



의료용 보조차 시장의 규모가 커지고 있음을 보여주는 그래프.

보조차 시장의 규모는 2021년에 비해 2028년에 약 800만 달러 증가할 것이라 예상됩니다.

이처럼 세계적으로 고령자의 수가 증가하면서 세계 의료용 보행기 시장 규모 또한 점점 커지고 있습니다.

기존의 단순한 기능만을 가지고 있는 보조차에 새로운 기능을 추가하고 사용자 맞춤 건강 보조 시스템을 만들어 고령 보행자의 교통사고 비율을 줄일 수 있습니다.

※기존 제품와의 비교

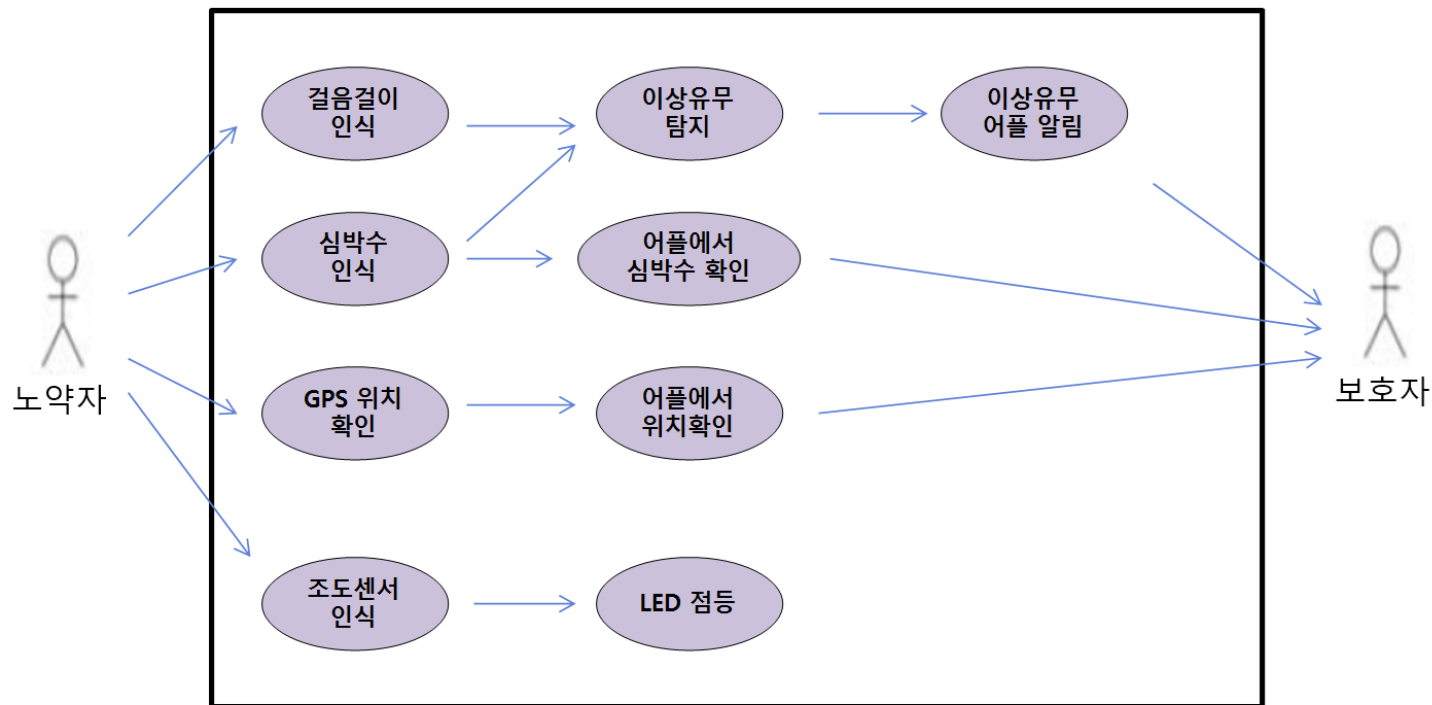
측정지표	기존 보행보조차	본 프로젝트 보행보조차
편리성	- 기본적인 이동 보조 기능 제공 - 주로 단순한 보행 지원 목적으로 사용	- 걸음걸이 이상 유무를 실시간으로 파악하여 사용자에게 피드백 제공 - 조도에 따른 LED 조명 기능으로 야간 및 어두운 환경에서도 안전하게 보행 가능
기능성	- 간단한 물리적 지원 (예: 손잡이, 바퀴 등) - 특정 상황에서는 사용자에게 제한적 지원	- 걸음걸이 분석을 통한 이상 유무 판단 - 조도 센서를 통해 환경에 맞는 LED 조명 자동 조절 - 실시간 데이터를 통한 사용자 맞춤형 피드백 제공 - GPS 및 긴급 호출 기능을 통해 사고 발생 시 신속한 대응 가능 - 사용자의 심박수를 모니터링하여 필요 시 경고 신호 발송
사용성	- 직관적인 설계와 간단한 조작법	- 다양한 기능을 통합하여 다양한 환경과 상황에서 효과적으로 사용 가능

요구사항 ID	요구사항 명	기능 ID	기능명	세부사항	비고
R01	실시간 건강 모니터링 시스템	R01-01	조도 측정	BH1750 센서를 통해 실시간으로 조도를 측정하고 데이터 수집	
		R01-02	오일러 각도 측정	IHARS 센서를 통해 오일러 각도(roll,pitch,yaw)를 측정	
		R01-03	심박수 측정	MCP3008을 통해 맥박 센서 데이터를 수집	
		R01-04	현재 위치 측정	neo -6M GPS 센서를 통해 실시간으로 사용자의 위치를 측정	
R02	데이터 전송 및 저장	R02-01	실시간 데이터 전송	수집된 데이터를 실시간으로 Firebase Firestore에 전송	
		R02-02	데이터 통합 저장	각 센서로부터 수집된 데이터를 통합하여 Firestore에 저장	
R03	데이터 처리 및 예측	R03-01	데이터 전처리	수집된 데이터를 TensorFlow 모델에 적합하게 전처리	
		R03-02	건강 이상 예측	TensorFlow 모델을 사용하여 건강 상태를 예측	
R04	알림 시스템	R04-01	이상 감지 알림	예측 모델에서 건강 이상을 감지하면 실시간으로 알림 전송	
R05	안드로이드 어플 제작	R05-01	로그인 화면 표시	제품 번호 등록과 로그인 화면 표시	
		R05-02	현재 위치 표시	neo-6M GPS 센서로부터 받은 위치 데이터를 앱에 실시간으로 표시	
		R05-03	푸시 알림 수신	Firebase Cloud Messaging을 사용하여 이상 감지 시 푸시 알림 수신 및 표시	
		R05-04	상세 화면 표시	Firestore에서 실시간 데이터를 가져와 사용자에게 표시	

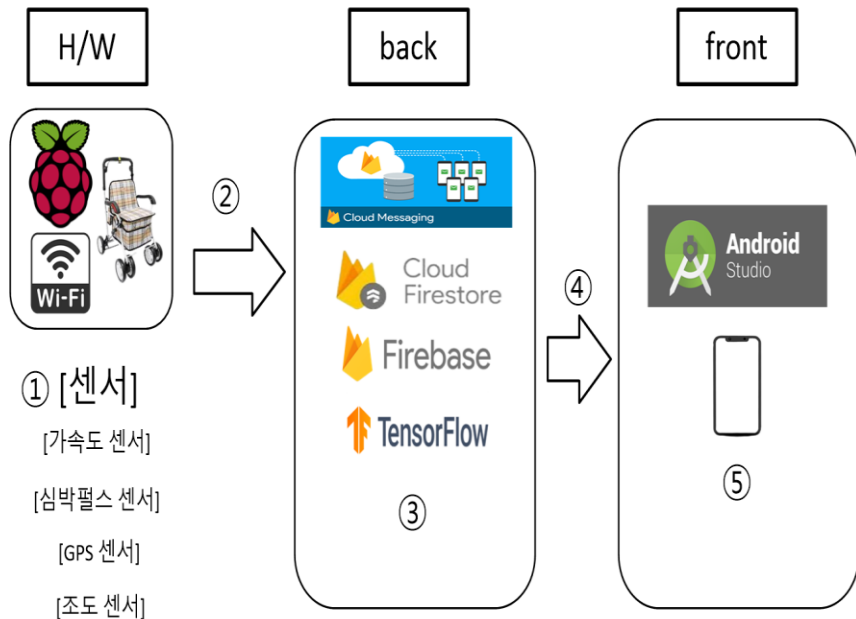
| 요구사항 정의서

구분	기능	설명
H/W	LED 제어	조도가 낮은 어두운 환경에서 LED 자동으로 점등
	충격 감지	사고 시 충격을 감지하여 긴급상황 알림
	가속도 센서	오일러 값을 통해 사용자의 걸음걸이 측정
	심박수 측정	사용자의 심박수를 실시간으로 모니터링
	gps 센서	실시간 위치 추적 및 긴급 상황시 빠른 대처 위한 위치 정보 제공
S/W	Firebase	사용자 인증 및 권한 관리
	Firebase Firestore	센서에서 받은 데이터 저장 및 관리
	Firebase Cloud Messaging	푸시 알림 전송
	TensorFlow	파이어베이스와 연동 후 머신러닝 모델을 사용하여 걸음걸이 패턴 분석 및 이상감지
	안드로이드 스튜디오	안드로이드 앱 제작

| 유즈케이스



| 서비스 구성도 - 서비스 시나리오



[H/W]

- ① 라즈베리파이와 각 센서를 이용하여 정보 수집
 - 가속도 센서 : 오일러 값을 이용하여 걸음걸이 측정
 - 심박펄스 센서 : 심박수 측정
 - GPS 센서 : 현재 위치의 경도 위도 값 측정
 - 조도 센서 : 조도 측정

- ② 센서 데이터를 수집, 이를 처리하여 Firebase에 전송

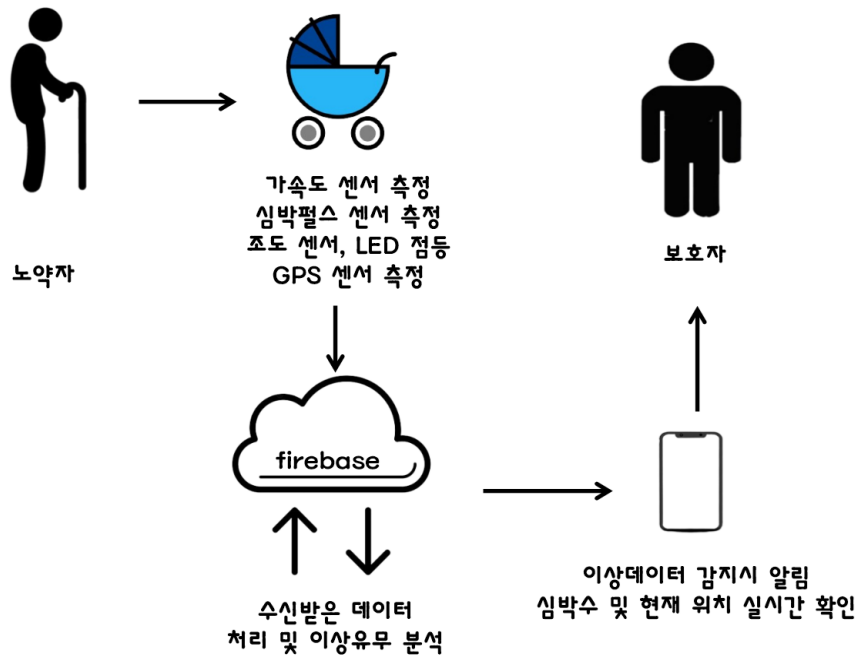
[back-end]

- ③ Firestore: 센서 데이터 저장 및 관리
 Firebase: 사용자 인증 및 권한 관리
 Firebase Cloud Messaging: 건강 이상이 감지 되었을 때 알림 전송
 TensorFlow: 수집된 센서 데이터를 기반으로 건강 상태 예측

[front-end]

- ⑤ 안드로이드 앱: 실시간 데이터 표시, 건강 이상 시 사용자에게 알림 제공

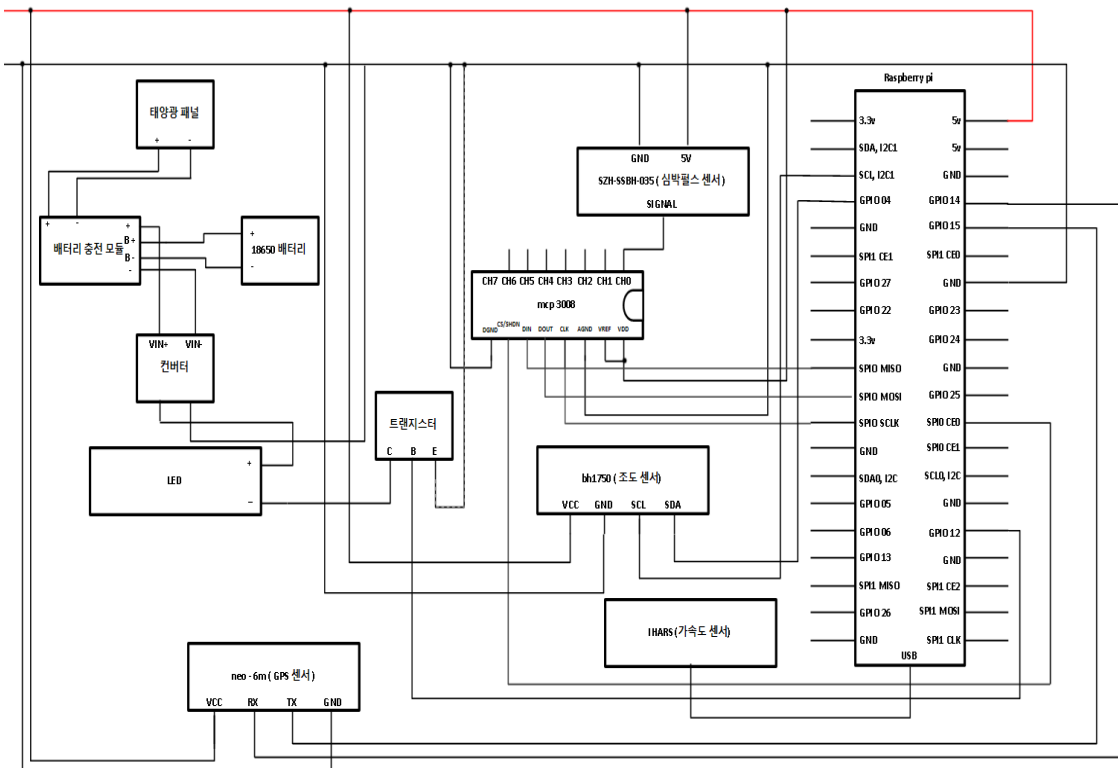
| 서비스 흐름도



[노약자가 보조차를 이용]

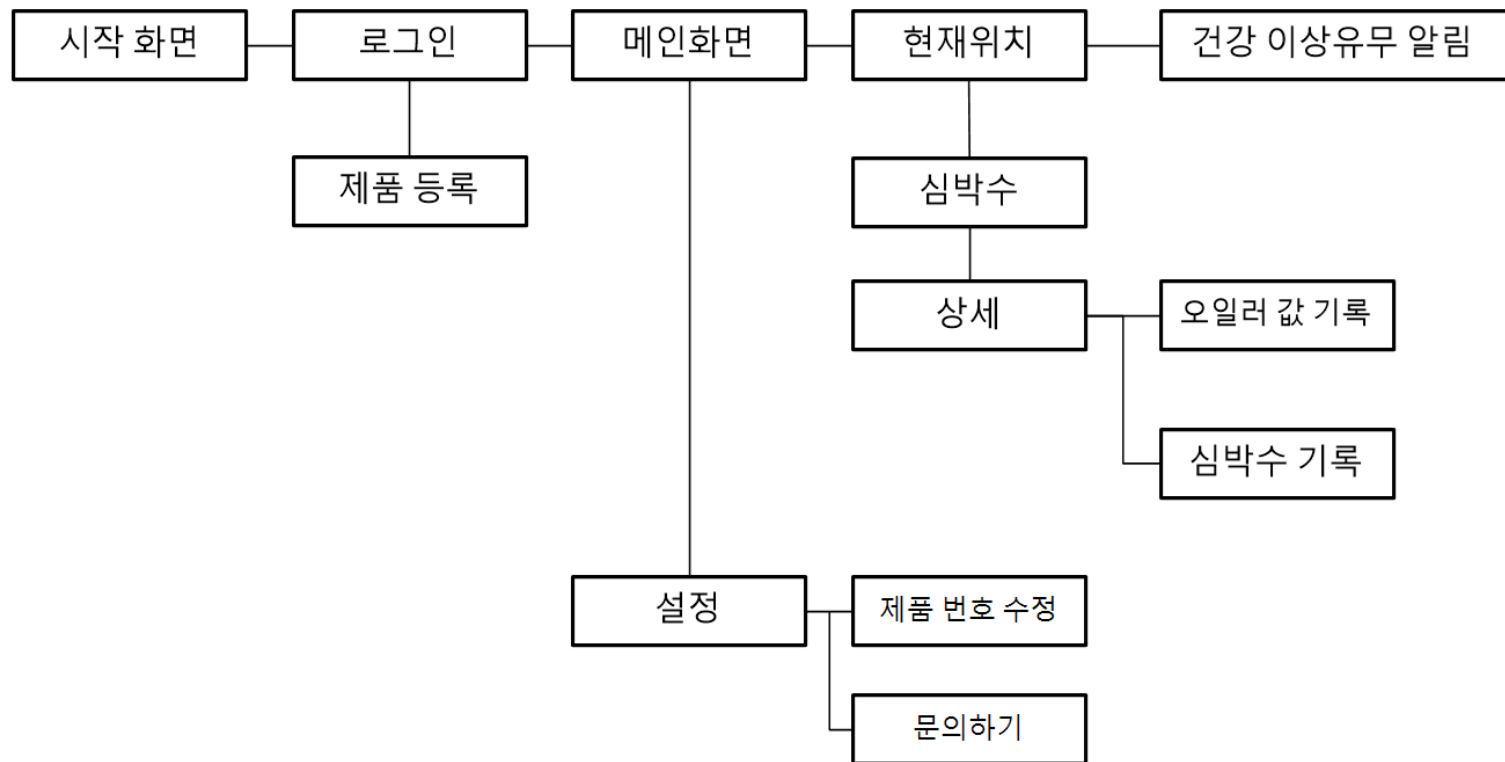
1. 각 센서가 데이터 값을 측정
1. 빛이 없는 곳에서 LED 점등
1. 측정 한 값을 firebase로 전송
1. firebase 안에서 tensorflow를 이용해 이상유무 탐지
1. 이상 데이터 탐지 시 어플로 알림 전송
1. 어플로 심박수 실시간 확인
1. 지도 API를 이용하여 어플에서 실시간 위치 확인

| 하드웨어/센서 구성도

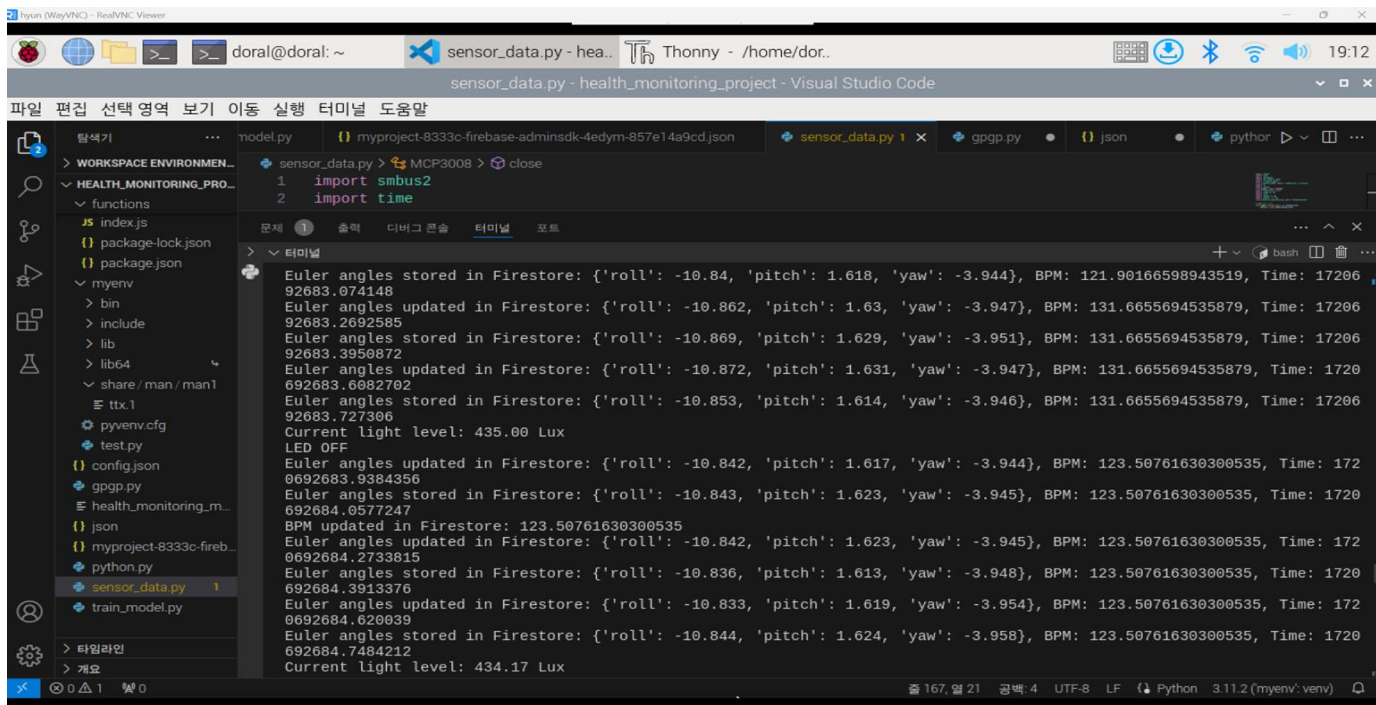


센서 종류	연결 핀	설명
BH 1750 (조도 센서)	GND	라즈베리파이의 GND에 연결
	VCC	라즈베리파이의 5V에 연결
	SCL	라즈베리파이의 SCL 핀에 연결
	SDA	라즈베리파이의 SDA 핀에 연결
SZH-SSBH-035 (심박펄스 센서)	GND	라즈베리파이의 GND에 연결
	VCC	라즈베리파이의 5V에 연결
	SIGNAL	mcp 3008(컨버터)의 CH0에 연결
neo - 6m (GPS 센서)	VCC	라즈베리파이의 5V에 연결
	GND	라즈베리파이의 GND에 연결
	RX	라즈베리파이의 GPIO 14번 핀에 연결
	TX	라즈베리파이의 GPIO 15번 핀에 연결
IHARS (가속도 센서)	USB	라즈베리파이의 USB 포트와 연결

| 메뉴 구성도



| 화면 설계서 s/w



```

sensor_data.py
1 import smbus2
2 import time

Euler angles stored in Firestore: {'roll': -10.84, 'pitch': 1.618, 'yaw': -3.944}, BPM: 121.90166598943519, Time: 17206
92683.074148
Euler angles updated in Firestore: {'roll': -10.862, 'pitch': 1.63, 'yaw': -3.947}, BPM: 131.6655694535879, Time: 17206
92683.2692585
Euler angles stored in Firestore: {'roll': -10.869, 'pitch': 1.629, 'yaw': -3.951}, BPM: 131.6655694535879, Time: 17206
92683.3950872
Euler angles updated in Firestore: {'roll': -10.872, 'pitch': 1.631, 'yaw': -3.947}, BPM: 131.6655694535879, Time: 1720
692683.6082702
Euler angles stored in Firestore: {'roll': -10.853, 'pitch': 1.614, 'yaw': -3.946}, BPM: 131.6655694535879, Time: 17206
92683.727306
Current light level: 435.00 Lux
LED OFF
Euler angles updated in Firestore: {'roll': -10.842, 'pitch': 1.617, 'yaw': -3.944}, BPM: 123.50761630300535, Time: 172
0692683.9384356
Euler angles stored in Firestore: {'roll': -10.843, 'pitch': 1.623, 'yaw': -3.945}, BPM: 123.50761630300535, Time: 1720
692684.0577247
BPM updated in Firestore: 123.50761630300535
Euler angles updated in Firestore: {'roll': -10.842, 'pitch': 1.623, 'yaw': -3.945}, BPM: 123.50761630300535, Time: 172
0692684.2733815
Euler angles stored in Firestore: {'roll': -10.836, 'pitch': 1.613, 'yaw': -3.948}, BPM: 123.50761630300535, Time: 1720
692684.3913376
Euler angles updated in Firestore: {'roll': -10.833, 'pitch': 1.619, 'yaw': -3.954}, BPM: 123.50761630300535, Time: 172
0692684.620039
Euler angles stored in Firestore: {'roll': -10.844, 'pitch': 1.624, 'yaw': -3.958}, BPM: 123.50761630300535, Time: 1720
692684.7484212
Current light level: 434.17 Lux
  
```

라즈베리파이 내에서 sensor_data.py 코드를 실행하여 센서를 통해 오일러 값, 속도, 심박수를 받아 오는 모습입니다.

| 화면 설계서 안드로이드앱

개발 예정

기능 번호	R05-01
기능 명	로그인 / 제품번호
기능설명	앱의 첫 화면 보조차 제품번호를 등록 보조차 제품번호로 로그인
처리내용	■ 보조차의 제품번호를 등록한다. 자신이 소지하고 있는 보조차의 등록번호를 입력한다. (로그인 아이콘을 누르면 로그인 창으로 넘어감) ■ 등록된 제품번호를 입력한 후 다음 화면으로 넘어간다.
비고	■ 연결되는 기능 : 제품번호 연동
요구사항 명	제품번호 등록 및 로그인

| 화면 설계서

개발 예정

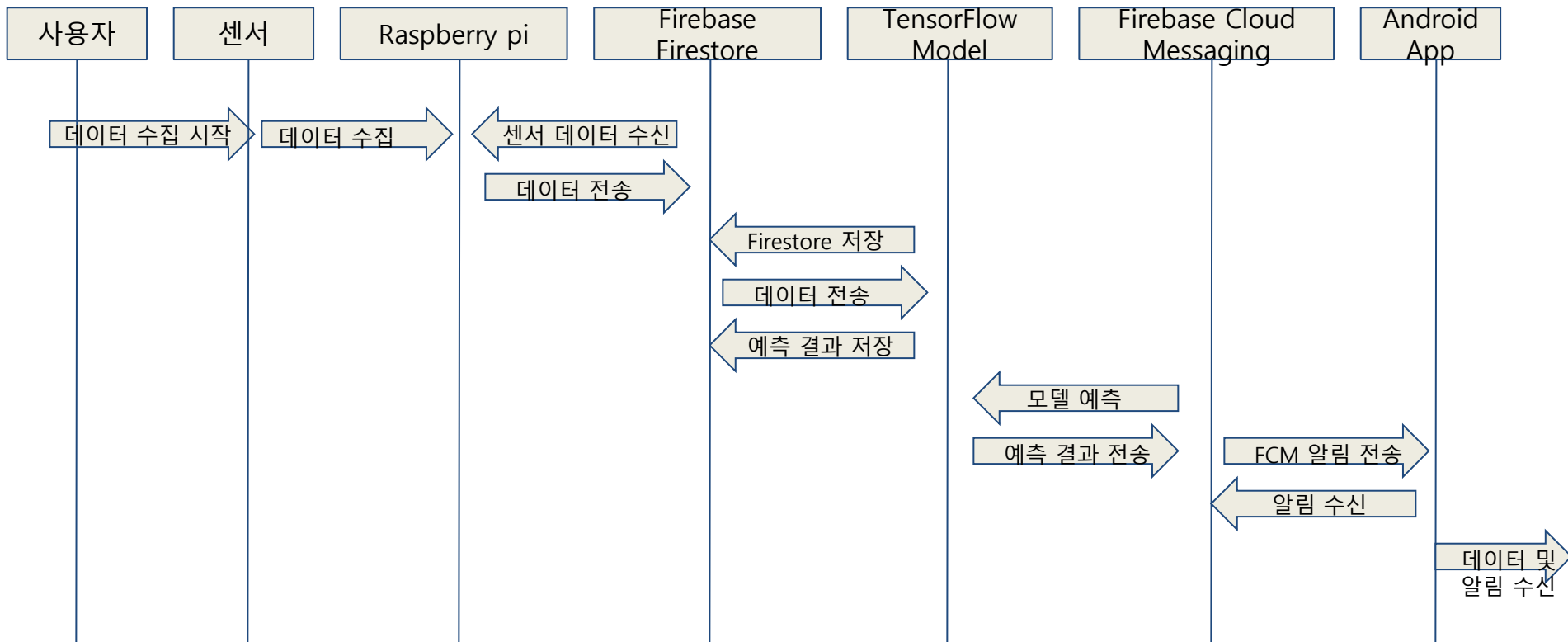
기능 번호	R05-02, R05-03
기능 명	메인 화면
기능설명	앱의 메인화면 보조차 현재 위치 실시간 표시 보조차를 사용하고있는 사용자의 심박수 실시간 표현 보조차를 사용하고있는 사용자의 건강 이상 발생 시 알림
처리내용	■ 지도 API를 사용하여 보조차의 현재 위치를 실시간으로 지도에 표시한다. Firebase에서 경도 위도값을 실시간으로 받아와 화면을 업데이트 ■ 현재 보조차를 사용하고있는 사용자의 실시간 심박수를 표시한다. Fire Store에서 현재 심박수를 실시간으로 받아와 화면에서 표시 ■ 현재 보조차를 사용하고있는 사용자의 건강 이상 발생 시 알림 표시한다. 이상유무 탐지 머신러닝 모델을 사용하여 건강 이상 데이터 감지 시 어플에서 알림
비고	■연결되는 기능 : Fire Store 연동, 알림
요구사항 명	현재위치 표시 및 심박수 표시

| 화면 설계서

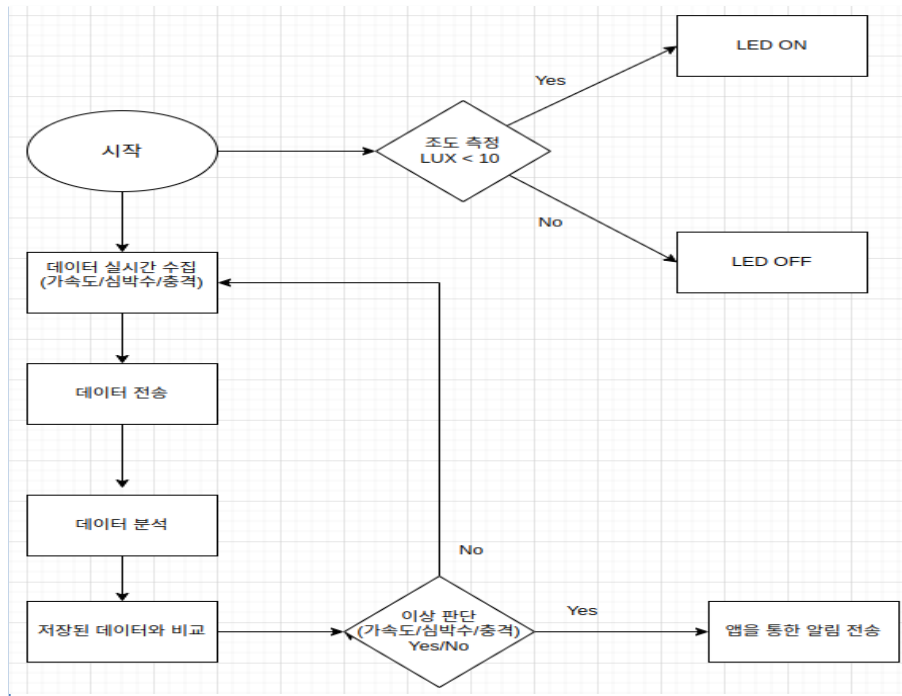
개발 예정

기능 번호	R05-04
기능 명	상세
기능설명	전제적인 상세 내용 표시 이전 오일러 값부터 현재 오일러 값까지 표현 이전 심박수부터 현재 심박수 까지 표현
처리내용	■ 이전에 측정한 오일러 값의 기록과, 현재 측정된 오일러 값을 화면에서 표현한다. 지금까지 Fire Store에서 받아온 오일러 값들을 저장하여 한번에 표시 ■ 이전에 측정한 심박수 값의 기록과, 현재 측정된 심박수 값을 화면에서 표현한다. 지금까지 Fire Store에서 받아온 심박수 값들을 저장하여 한번에 표시
비고	■연결되는 기능 : 파이어 스토어 연동
요구사항 명	오일러 값 표시 및 심박수 표시

| 기능 처리도(기능 흐름도)



| 알고리즘 명세서



- ①시스템이 시작되면 보행 보조차의 가속도, 심박수, 충격 데이터 실시간 수집과 조도측정을 시작한다.
- ②가속도 데이터를 통해 걸음걸이 이상 여부를 판단한다.
- ③심박수 데이터를 통해 심박수 이상 여부를 판단한다.
- ④충격 데이터를 통해 충격 발생 여부를 판단한다.
- ⑤데이터 분석 도중 저장된 정상 데이터와 수집된 데이터를 비교한다.
- ⑥비교 결과에서 이상이 감지될 경우:
걸음걸이,심박수,충격데이터에서 이상이 감지되면 보호자에게 알림을 보낸다.
- ⑦이상이 감지되지 않을 경우, 시스템은 계속해서 실시간 데이터를 수집하고 분석을 반복한다.
- ⑧실시간 데이터 수집과 분석이 계속되는 동안, 이상 감지 시 보호자에게 즉시 알림을 보낸다
- ⑨시스템이 종료되지 않고, 계속해서 데이터 수집 및 분석 과정을 반복한다.
- ⑩조도가 10LUX이하로 측정될 경우 LED를 점등한다.

| 알고리즘 상세 설명서

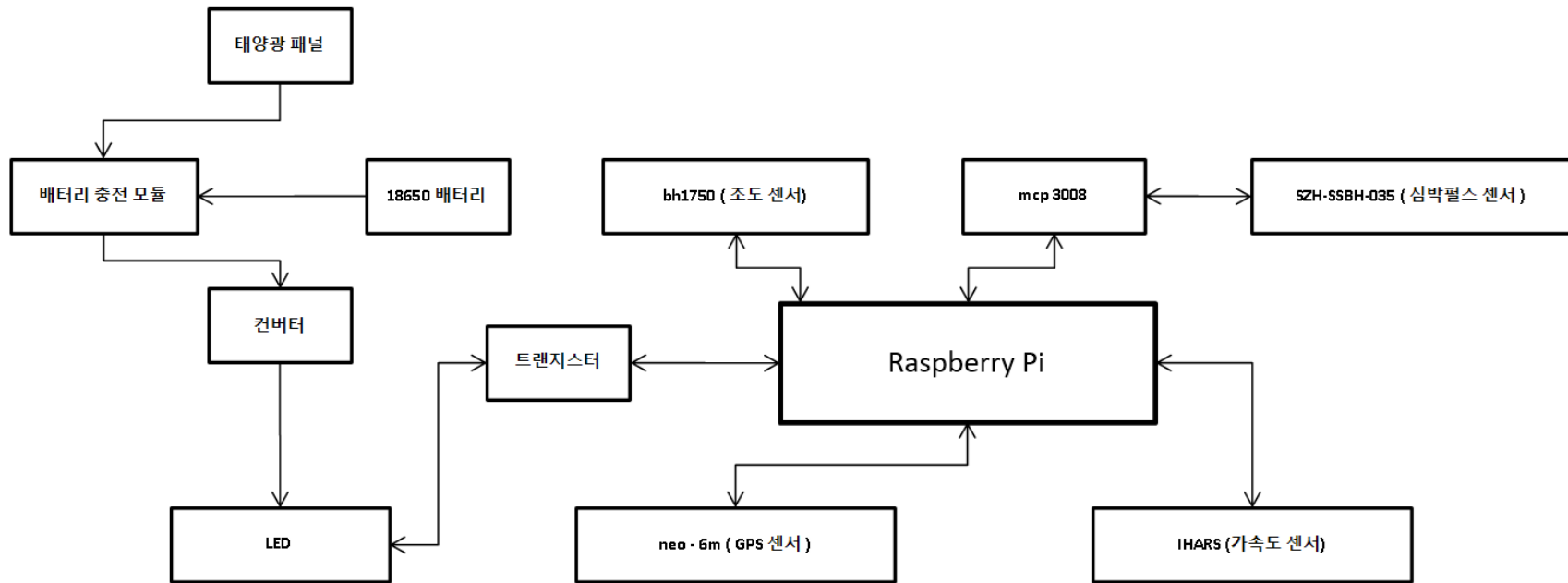
걸음걸이 이상 탐지 알고리즘은 IHARS 가속도 센서를 사용하여 실시간으로 걸음걸이 데이터를 수집하고 분석합니다. 이 알고리즘은 수집된 가속도 데이터를 기반으로 걸음걸이 패턴을 분석하여 이상 유무를 판단합니다. 만약 걸음걸이 패턴에서 이상이 감지되면, 이를 사용자에게 알리기 위해 Firebase Cloud Messaging을 통해 실시간 알림을 보냅니다.

이 알고리즘의 주요 단계는 다음과 같습니다

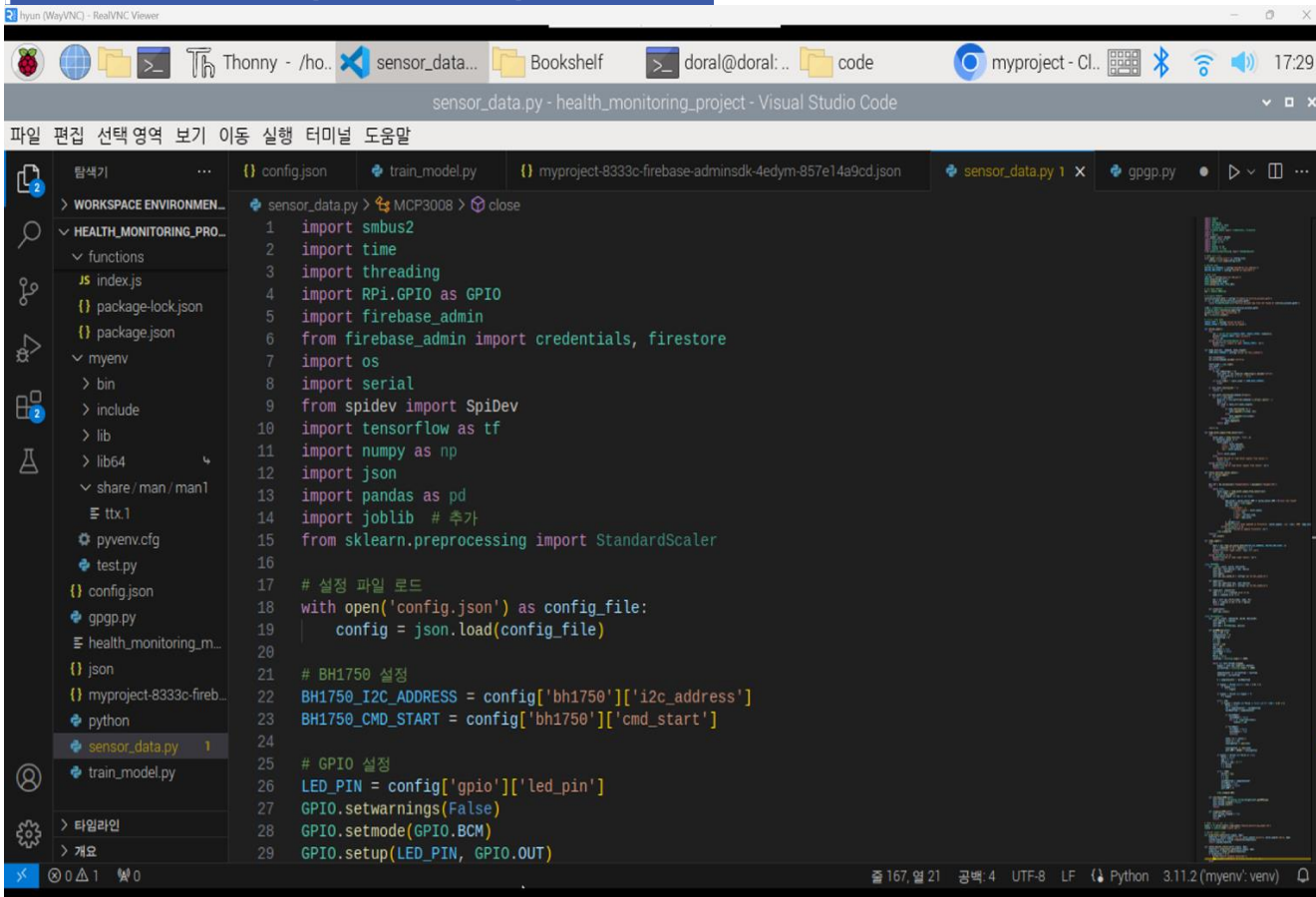
1. **데이터 수집**: IHARS 가속도 센서가 실시간으로 가속도 데이터를 수집합니다.
2. **데이터 전처리**: 수집된 데이터를 필요한 형식으로 전처리합니다.
3. **특징 추출**: 걸음걸이 패턴을 나타내는 주요 특징을 추출합니다.
4. **모델 예측**: TensorFlow 모델을 사용하여 정상 및 비정상 걸음걸이 패턴을 예측합니다.
5. **알림 전송**: 비정상 패턴이 감지되면 Firebase Cloud Messaging을 통해 실시간 알림을 전송합니다.

데이터 수집 단계에서는 IHARS 가속도 센서가 실시간으로 걸음걸이 데이터를 수집합니다. 이 데이터를 라즈베리 파이가 수신하고, 전처리 과정을 통해 노이즈 제거 및 스케일링을 수행하여 분석에 적합한 형식으로 변환합니다.

| 하드웨어 설계도



핵심소스코드(전체 모습)



```

1 import smbus2
2 import time
3 import threading
4 import RPi.GPIO as GPIO
5 import firebase_admin
6 from firebase_admin import credentials, firestore
7 import os
8 import serial
9 from spidev import SpiDev
10 import tensorflow as tf
11 import numpy as np
12 import json
13 import pandas as pd
14 import joblib # 추가
15 from sklearn.preprocessing import StandardScaler
16
17 # 설정 파일 로드
18 with open('config.json') as config_file:
19     config = json.load(config_file)
20
21 # BH1750 설정
22 BH1750_I2C_ADDRESS = config['bh1750']['i2c_address']
23 BH1750_CMD_START = config['bh1750']['cmd_start']
24
25 # GPIO 설정
26 LED_PIN = config['gpio']['led_pin']
27 GPIO.setwarnings(False)
28 GPIO.setmode(GPIO.BCM)
29 GPIO.setup(LED_PIN, GPIO.OUT)
  
```

라즈베리파이 내에서 vscode를 사용하여 sensor_data.py 및 config.json, 이후 추가 예정인 모델학습에 필요한 train_model.py 파일을 Healthy_Monitoring_project 폴더 안에 만들어 놓은 모습입니다.

위에 function 폴더에는 코드 동작에 필요한 파일들을 설치해놓은 모습입니다.

핵심소스코드(센서와 모듈 설정)

```
{ } config.json > { } firebase > service_account_path
1  {
2      "bh1750": {
3          "i2c_address": 35,
4          "cmd_start": 16
5      },
6      "gpio": {
7          "led_pin": 12
8      },
9      "firebase": { }
10     "service_account_path": "/home/doral/myproject-8333c-firebase-adminsdk-4edym-cc505accf3.json"
11 },
12 "serial": {
13     "port": "/dev/ttyUSB0",
14     "speed": 115200,
15     "recv_timeout": 30
16 },
17 "spi": {
18     "bus": 0,
19     "device": 0,
20     "max_speed_hz": 1000000
21 }
22 }
```

이 설정 파일을 통해 코드가 실행될 때 각 모듈이 올바르게 초기화되고 구성될 수 있습니다. BH1750 조도 센서는 지정된 I2C 주소와 명령으로 초기화되며, LED는 지정된 GPIO 핀에 연결되어 제어됩니다.

Firebase는 서비스 계정 키 파일을 사용하여 인증되며, 시리얼 포트와 SPI는 각각 지정된 설정으로 초기화됩니다.

|핵심소스코드(조도센서,LED)

```
# BH1750 설정
```

```
BH1750_I2C_ADDRESS = config['bh1750']['i2c_address']
```

```
BH1750_CMD_START = config['bh1750']['cmd_start']
```

```
def read_light():
```

```
    try:
```

```
        data = bus.read_i2c_block_data(BH1750_I2C_ADDRESS, BH1750_CMD_START, 2)
```

```
        lux = ((data[0] << 8) + data[1]) / 1.2
```

```
        print(f"Current light level: {lux:.2f} Lux")
```

```
        return lux
```

```
    except Exception as e:
```

```
        print(f"Failed to read light sensor: {e}")
```

```
        return None
```

`config.json` 파일에서 BH1750 조도 센서의 I2C 주소와 시작 명령을 읽어옵니다.

`read_light` 함수는 조도 센서로부터 데이터를 읽어오는 함수입니다.

`bus.read_i2c_block_data` 함수는 지정된 I2C 주소에서 데이터를 읽습니다.

읽어온 두 바이트의 데이터를 합쳐서 조도 값을 계산합니다. BH1750 센서의 출력 값은 Lux 단위로 변환됩니다. 읽어온 Lux 값을 출력하고 반환합니다.

예외가 발생하면 에러 메시지를 출력하고 `None`을 반환합니다.

|핵심소스코드(조도센서,LED)

```
def store_realtime_sensor_data():
    ser = serial_open()
    if not ser:
        return

    doc_ref = db.collection(u'HomeAutomatic').document(u'RaspberryPi')
    try:
        while True:
            euler_angle = read_euler_angle_from_sensor(ser)
            lux = read_light() # 여기에서 조도 센서 데이터를 읽습니다.
            if euler_angle and lux is not None:
                try:
                    bpm_value = pulse_sensor.BPM if pulse_sensor.BPM > 0 else "not found"
                    current_time = time.time()
                    doc_ref.set({
                        u'SensorData': {
                            u'euler_angle': euler_angle,
                            u'lux': lux,
                            u'time': current_time,
                            u'bpm': bpm_value
                        }
                    }, merge=True)
                    print(f"Sensor data updated in Firestore: {euler_angle}, Lux: {lux},")
                except Exception as e:
                    print(f"Failed to update Firestore: {e}")
            time.sleep(10)
    finally:
        ser.close()
```

store_realtime_sensor_data 함수에서 read_light 함수를 호출하여 조도 센서 데이터를 읽습니다. 읽어온 조도 센서 데이터를 Firestore에 저장합니다.

|핵심소스코드(심박펄스 센서)

```
class MCP3008:
    def __init__(self, bus=0, device=0):
        self.bus, self.device = bus, device
        self.spi = SpiDev()
        self.open()
        self.spi.max_speed_hz = config['spi']['max_speed_hz']

    def open(self):
        self.spi.open(self.bus, self.device)
        self.spi.max_speed_hz = config['spi']['max_speed_hz']

    def read(self, channel=0):
        cmd1 = 4 | 2 | ((channel & 4) >> 2)
        cmd2 = (channel & 3) << 6

        adc = self.spi.xfer2([cmd1, cmd2, 0])
        data = ((adc[1] & 15) << 8) + adc[2]
        return data

    def close(self):
        self.spi.close()
```

심박 펄스 데이터를 읽기 위해 MCP3008 아날로그-디지털 변환기를 사용합니다.
MCP3008 아날로그-디지털 변환기를 제어합니다.
`read` 메서드는 지정된 채널에서 아날로그 데이터를 읽어 디지털 값으로 변환합니다.

|핵심소스코드(심박펄스 센서)

```
class Pulsesensor:
    def __init__(self, channel=0, bus=0, device=0):
        self.channel = channel
        self.BPM = 0
        self.adc = MCP3008(bus, device)

    def getBPMLoop(self):
        rate = [0] * 10
        sampleCounter = 0
        lastBeatTime = 0
        P = 512
        T = 512
        thresh = 525
        amp = 100
        firstBeat = True
        secondBeat = False
        IBI = 600
        Pulse = False
        lastTime = int(time.time() * 1000)

        while not self.thread.stopped:
            Signal = self.adc.read(self.channel)
            currentTime = int(time.time() * 1000)

            sampleCounter += currentTime - lastTime
            lastTime = currentTime

            N = sampleCounter - lastBeatTime
```

```
        if Signal < thresh and N > (IBI / 5.0) * 3:
            if Signal < T:
                T = Signal

            if Signal > thresh and Signal > P:
                P = Signal

        if N > 250:
            if Signal > thresh and Pulse is False and N > (IBI / 5.0) * 3:
                Pulse = True
                IBI = sampleCounter - lastBeatTime
                lastBeatTime = sampleCounter

            if secondBeat:
                secondBeat = False
                for i in range(len(rate)):
                    rate[i] = IBI

            if firstBeat:
                firstBeat = False
                secondBeat = True
                continue

        rate[:-1] = rate[1:]
        rate[-1] = IBI
        runningTotal = sum(rate)
```

| 핵심소스코드(GPS)

```
if Signal < thresh and Pulse == True:
    Pulse = False
    amp = P - T
    thresh = amp / 2 + T
    P = thresh
    T = thresh

if N > 2500:
    thresh = 512
    P = 512
    T = 512
    lastBeatTime = sampleCounter
    firstBeat = True
    secondBeat = False
    self.BPM = 0

    time.sleep(0.005)

def startAsyncBPM(self):
    self.thread = threading.Thread(target=self.getBPMLoop)
    self.thread.stopped = False
    self.thread.start()
    return

def stopAsyncBPM(self):
    self.thread.stopped = True
    self.BPM = 0
    return
```

Pulsesensor 클래스는 심박 펄스를 측정하고 BPM(분당 박동 수)을 계산합니다.

`__init__` 메서드는 MCP3008 아날로그-디지털 변환기의 인스턴스를 초기화합니다.

`getBPMLoop` 메서드는 심박 펄스 신호를 지속적으로 읽고 BPM을 계산하는 비동기 루프입니다.

`startAsyncBPM` 메서드는 `getBPMLoop`을 비동기적으로 실행하는 스레드를 시작합니다.

`stopAsyncBPM` 메서드는 비동기 스레드를 중지합니다.

| 핵심소스코드(GPS)

```
def store_realtime_sensor_data():
    ser = serial_open()
    if not ser:
        return

    doc_ref = db.collection(u'HomeAutomatic').document(u'RaspberryPi')
    try:
        while True:
            euler_angle = read_euler_angle_from_sensor(ser)
            lux = read_light()
            if euler_angle and lux is not None:
                try:
                    bpm_value = pulse_sensor.BPM if pulse_sensor.BPM > 0 else "not found"
                    current_time = time.time()
                    doc_ref.set({
                        u'SensorData': {
                            u'euler_angle': euler_angle,
                            u'lux': lux,
                            u'time': current_time,
                            u'bpm': bpm_value
                        }
                    }, merge=True)
                    print(f"Sensor data updated in Firestore: {euler_angle}, Lux: {lux},")
                except Exception as e:
                    print(f"Failed to update Firestore: {e}")
                    time.sleep(10)
    finally:
        ser.close()
```

```
if __name__ == "__main__":
    threading.Thread(target=store_realtime_sensor_data).start()
    threading.Thread(target=read_light).start()

    pulse_sensor = PulseSensor()
    pulse_sensor.startAsyncBPM()

    try:
        while True:
            monitor_health()
            time.sleep(1)
    except KeyboardInterrupt:
        pulse_sensor.stopAsyncBPM()
        GPIO.cleanup()
```

`store_realtime_sensor_data` 함수는 `pulse_sensor.BPM`을 사용하여 BPM 값을 Firestore에 저장합니다.

`bpm_value = pulse_sensor.BPM if pulse_sensor.BPM > 0 else "not found"` 줄에서 심박 펄스 데이터를 읽어옵니다.

`PulseSensor` 클래스의 인스턴스를 생성하고 `startAsyncBPM` 메서드를 호출하여 심박 펄스 측정을 비동기적으로 시작합니다.

프로그램 종료 시 `stopAsyncBPM` 메서드를 호출하여 심박 펄스 측정을 중지합니다.

이 코드에서 `PulseSensor` 클래스는 MCP3008 아날로그-디지털 변환기를 통해 심박펄스를 측정하고, 실시간으로 BPM 값을 계산하여 Firestore에 저장하는 역할을 합니다.

`getBPMLoop` 메서드는 심박 펄스 신호를 지속적으로 읽고 BPM을 계산하는 핵심 기능을 담당합니다.

| 핵심소스코드(가속도센서)

```
def read_euler_angle_from_sensor(ser):  
    try:  
        euler_data = send_recv(ser, "e\n", 3)  
        if len(euler_data) == 3:  
            euler_angle = {  
                'roll': euler_data[0],  
                'pitch': euler_data[1],  
                'yaw': euler_data[2]  
            }  
            return euler_angle  
        else:  
            print("Failed to read Euler angles from sensor.")  
            return None  
    except Exception as e:  
        print(f"Failed to read Euler angles from sensor: {e}")  
        return None
```

이 함수는 시리얼 포트를 통해 Euler 각도 데이터를 센서로부터 읽어옵니다.

`send_recv` 함수를 사용하여 "e\n" 명령을 보내고
응답으로 받은 데이터를 처리합니다.

응답 데이터가 3개의 값(roll, pitch, yaw)으로 구성되어
있으면 이를 `euler_angle` 딕셔너리로 반환합니다.

| 핵심소스코드(가속도센서)

```
def store_realtime_sensor_data():
    ser = serial_open()
    if not ser:
        return

    doc_ref = db.collection(u'HomeAutomatic').document(u'RaspberryPi')
    try:
        while True:
            euler_angle = read_euler_angle_from_sensor(ser)
            lux = read_light()
            if euler_angle and lux is not None:
                try:
                    bpm_value = pulse_sensor.BPM if pulse_sensor.BPM > 0 else "not found"
                    current_time = time.time()
                    doc_ref.set({
                        u'SensorData': {
                            u'euler_angle': euler_angle,
                            u'lux': lux,
                            u'time': current_time,
                            u'bpm': bpm_value
                        }
                    }, merge=True)
                    print(f"Sensor data updated in Firestore: {euler_angle}, Lux: {lux},")
                except Exception as e:
                    print(f"Failed to update Firestore: {e}")
            time.sleep(10)
    finally:
        ser.close()
```

`store_realtime_sensor_data` 함수에서 `read_euler_angle_from_sensor` 함수를 호출하여 Euler 각도 데이터를 읽어옵니다. 읽어온 Euler 각도 데이터를 Firestore에 저장합니다.

| 핵심소스코드(가속도센서)

보행자의 걸음걸이 이상 유무를 판단하기 위해 가속도 센서로 측정된 데이터를 Firebase로 보내고 TensorFlow를 이용해 딥러닝 모델로 분석하는 시스템을 구축합니다. 이 시스템은 MPU6050 가속도 센서를 통해 보행자의 걸음걸이 데이터를 실시간으로 측정합니다. 센서에서 얻은 가속도 데이터는 X, Y, Z 축의 값을 포함합니다. Firebase를 초기화하고 Firestore 데이터베이스에 연결하여 수집된 가속도 데이터를 실시간으로 Firestore에 업데이트합니다. 이를 통해 클라우드에서 데이터를 안전하게 저장하고, 언제든지 접근할 수 있도록 합니다. 가속도 데이터는 일정한 간격으로 Firebase Firestore에 업데이트되며, 지속적으로 데이터를 수집하고 네트워크를 통해 Firebase에 데이터를 전송하여 실시간 데이터 동기화를 유지합니다. 수집된 데이터를 이용하여 TensorFlow 딥러닝 모델을 학습시키며, 모델은 정상적인 보행 패턴과 이상 보행 패턴을 학습하여 실시간으로 들어오는 새로운 데이터를 기반으로 걸음걸이의 이상 유무를 판단합니다. 학습된 모델을 사용하여 실시간으로 들어오는 가속도 데이터를 분석하며, 이 분석을 통해 보행자의 걸음걸이에 이상이 있는지 여부를 판단합니다.

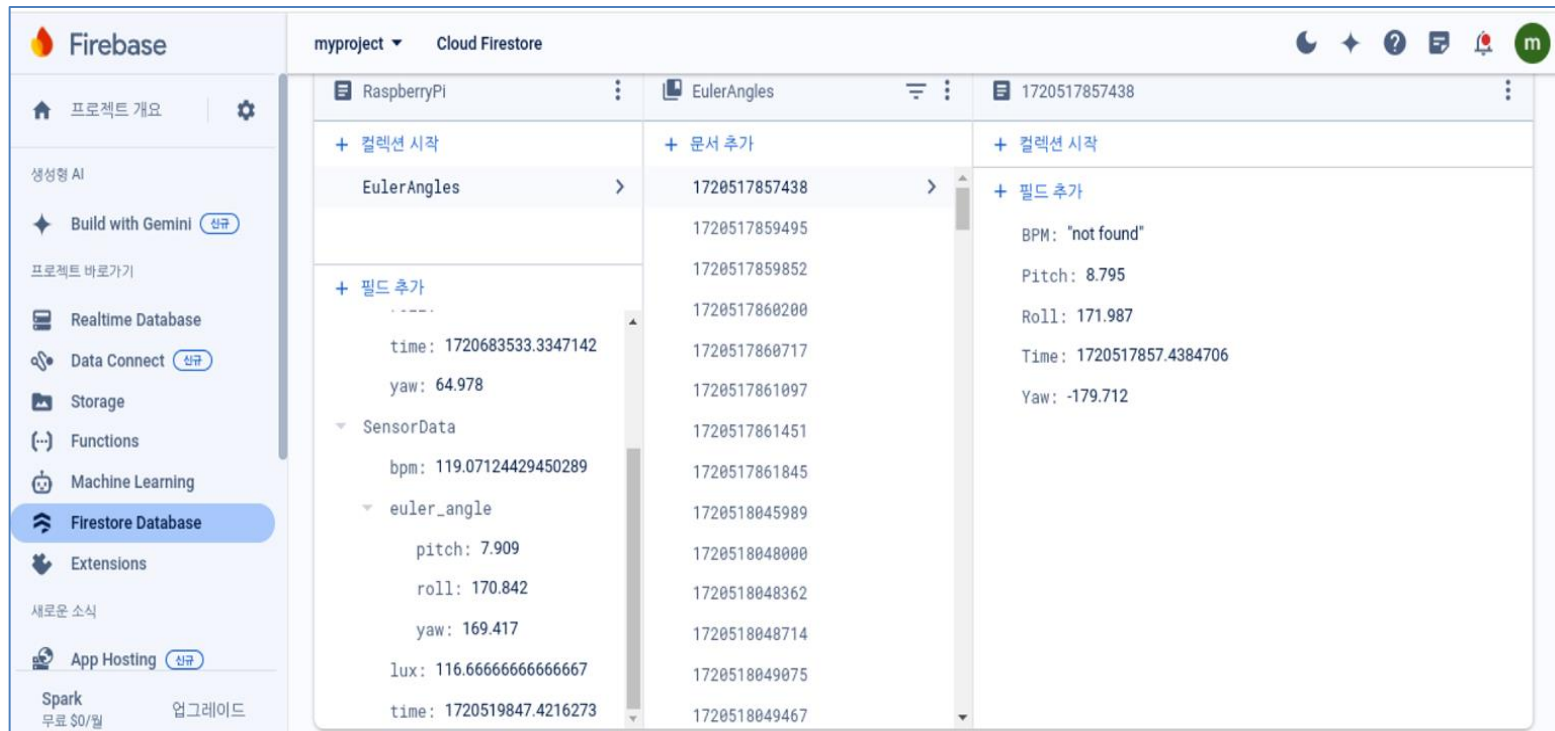
개발 예정

| 참조- 개발 환경 및 설명

구분		항목	적용내역
S/W 개발환경	스마트폰 App 개발	Android Studio	Android application 프로그램 개발
		안드로이드 OS	스마트폰 운영체제
	Firebase	Tensor Flow	데이터 학습 및 딥러닝 모델 구축
		FireStore	센서를 통해 얻은 데이터를 저장, 관리하는 데이터베이스
		Firebase Cloud Messaging	메시지와 알림 전송
H/W 구성장비	디바이스	스마트폰 (안드로이드)	사용자에게 서비스를 직접적으로 제공하는 End Device
		Raspberry Pi	각종 센서로 부터 값을 받고 전송
	통신	WIFI	Firebase와 통신
	기타	보조차	Raspberry Pi와 각종 센서를 부착

| 참조-S/W 기능 실사 사진

센서로 받은 오일러 값, 심박수, 빛의 세기를
Firestore DB로 실시간 전송 및 시간순으로 저장



The screenshot shows the Firebase Cloud Firestore interface. The left sidebar contains the project overview and various services. The main area displays the 'EulerAngles' collection under the 'myproject' database. The collection contains documents with the following data:

Document ID	time	yaw	bpm	euler_angle
1720517857438	1720683533.3347142	64.978	119.07124429450289	{pitch: 7.909, roll: 170.842, yaw: 169.417}
1720517859495				
1720517859852				
1720517860200				
1720517860717				
1720517861097				
1720517861451				
1720517861845				
1720518045989				
1720518048000				
1720518048362				
1720518048714				
1720518049075				
1720518049467				

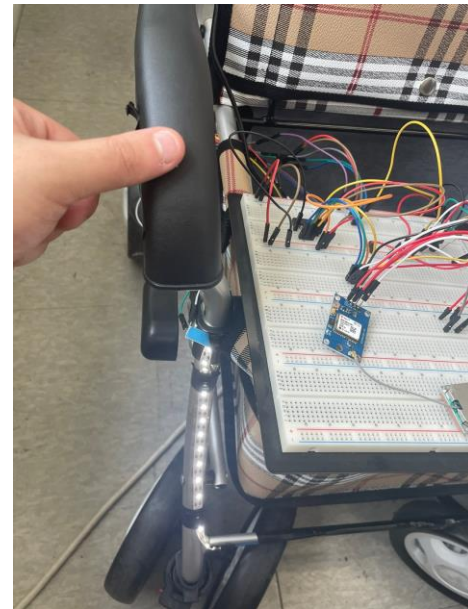
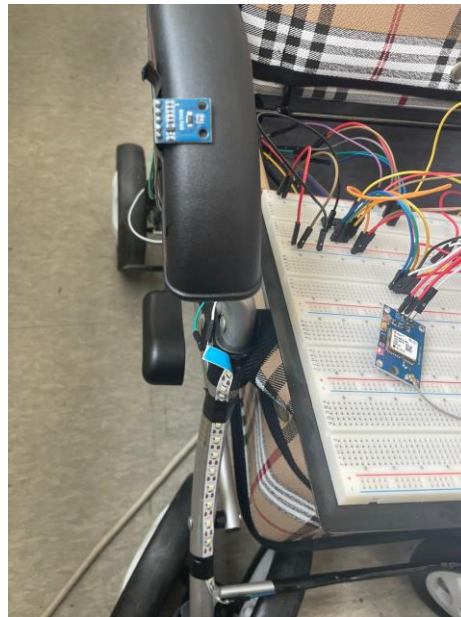
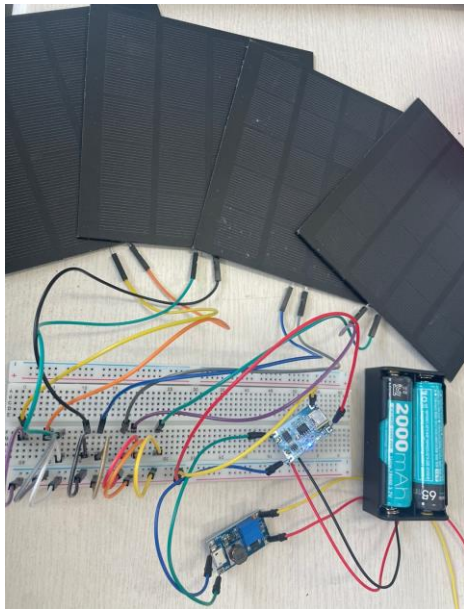
| 참조- H/W 기능 실사사진

보행 보조차 전체 모습



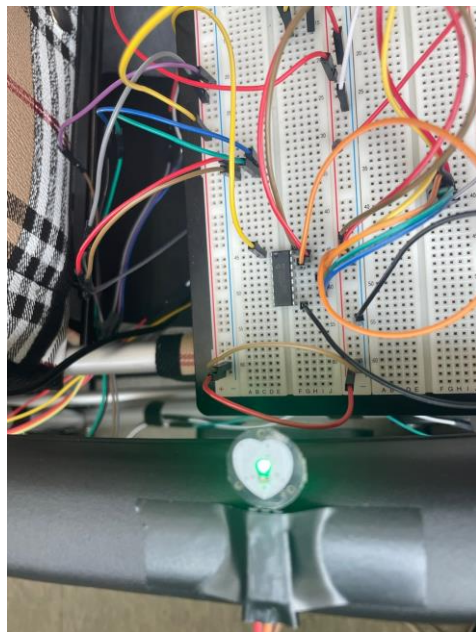
| 참조- H/W 기능 실사사진

태양광 패널 및 조도센서,LED



| 참조- H/W 기능 실사사진

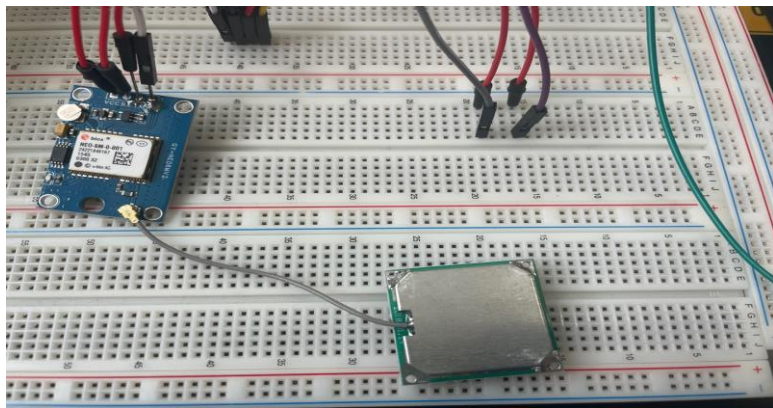
심박펄스 센서 및 심박수 측정 값



```
doral@doral:~/code $ python3 ggg.py  
No Heartbeat found  
No Heartbeat found  
No Heartbeat found  
No Heartbeat found  
No Heartbeat found  
No Heartbeat found  
No Heartbeat found  
BPM: 46.22496147919877  
BPM: 50.08347245409015  
BPM: 59.20078934385792  
BPM: 72.5689404934688  
BPM: 82.5082508250825
```

| 참조- H/W 기능 실사사진

GPS 모듈 및 현재 위치 측정



```
doral@doral:~/health_monitoring_project $ python3 qpgp.py
Time: 2024-07-11T06:21:39.000Z
Latitude: 37.210210323
Longitude: 126.975510502
Altitude: 61.4869
Time: 2024-07-11T06:21:40.000Z
Latitude: 37.210210323
Longitude: 126.975510502
Altitude: 61.4869
Time: 2024-07-11T06:21:40.000Z
Latitude: 37.210210323
Longitude: 126.975510502
Altitude: 61.4869
```

| 참조- H/W 기능 실사사진

가속도 센서 및 오일러 값 측정

```
^Cdoral@doral:~/code $ python3 olr.py
/dev/ttyUSB0 open success
Euler angle = -150.003, 15.506, 77.659
Quaternion = -0.118029, 0.767487, 0.572916, -0.262298
Euler angle = -149.978, 15.492, 77.661
Quaternion = -0.118327, 0.767443, 0.572897, -0.262332
Euler angle = -149.818, 15.383, 77.639
Quaternion = -0.120049, 0.767324, 0.57265, -0.262439
Euler angle = -149.813, 15.415, 77.643
Quaternion = -0.119841, 0.767325, 0.572595, -0.26265
Euler angle = -149.948, 15.506, 77.662
Quaternion = -0.118367, 0.767425, 0.572804, -0.262573
Euler angle = -149.895, 15.497, 77.662
Quaternion = -0.118877, 0.76735, 0.572703, -0.262779
Euler angle = -149.738, 15.379, 77.638
Quaternion = -0.118578, 0.767358, 0.572474, -0.262798
```



**Thank
you**

